



CS 498: Memory Optimization

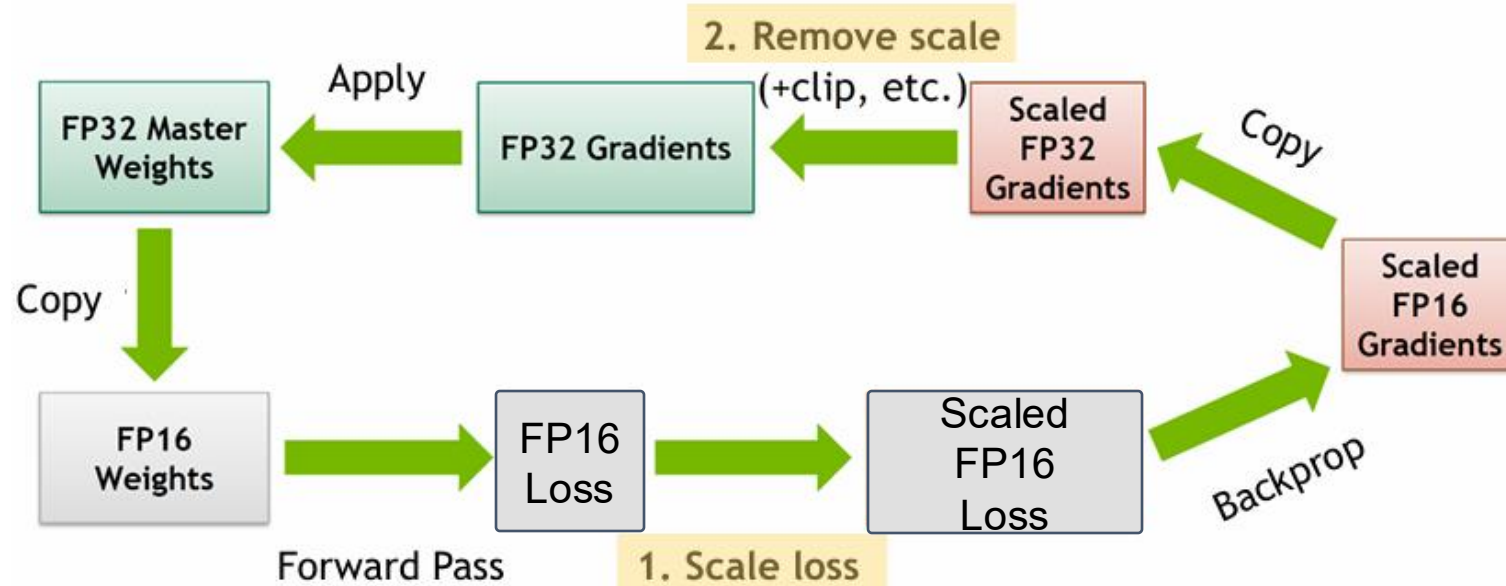
Fan Lai

The Grainger College of Engineering

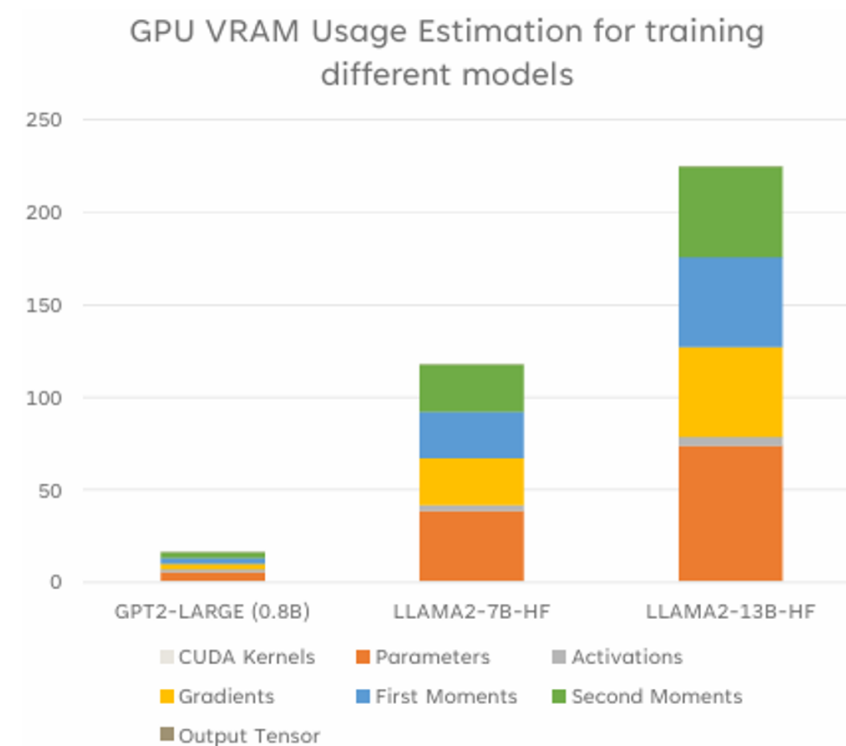
Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

- **Project proposal due today~**
 - Submit on Canvas (2 pages, plus more for references)
- **Sign up for feedback sessions on 10/08, 10/10, 10/15 in SC 3128**
 - No regular class on these days
 - Help you refine and plan the project (e.g., solution overview)
 - Sign up using this Google Sheet [[link](#)] (link available on Canvas too)

- **Mixed precision improves efficiency but may hurt quality** (FP16 vs. INT8)
- **Automatic Scaling at runtime may help quality**
 - E.g., scaling the loss to prevent underflow



- **Task:** Reduce the memory consumption to train a DL model on single device
- **Main challenge:** Cost to store intermediate results and gradients

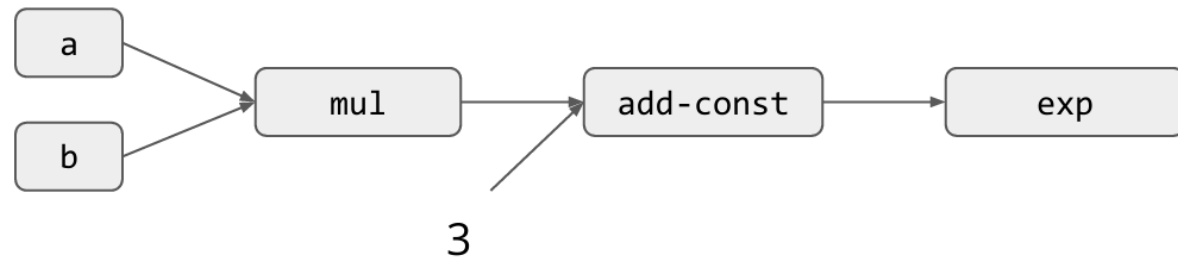


- **Task:** Reduce the memory consumption to train a DL model on single device
- **Main challenge:** Cost to store intermediate results and gradients
- **Key Ideas:**
 - Computation *graph* memory optimization (In-place Storing, Memory Sharing)
 - Save memory by re-computing (gradient checkpointing)

Build an Executor for a Given Graph



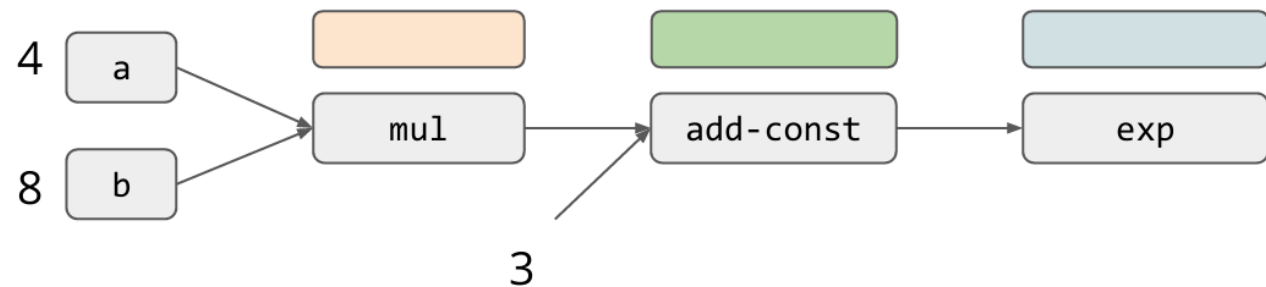
Computational Graph for $\exp(a * b + 3)$



1. **Allocate** temp memory for intermediate computation

Computational Graph for $\exp(a * b + 3)$

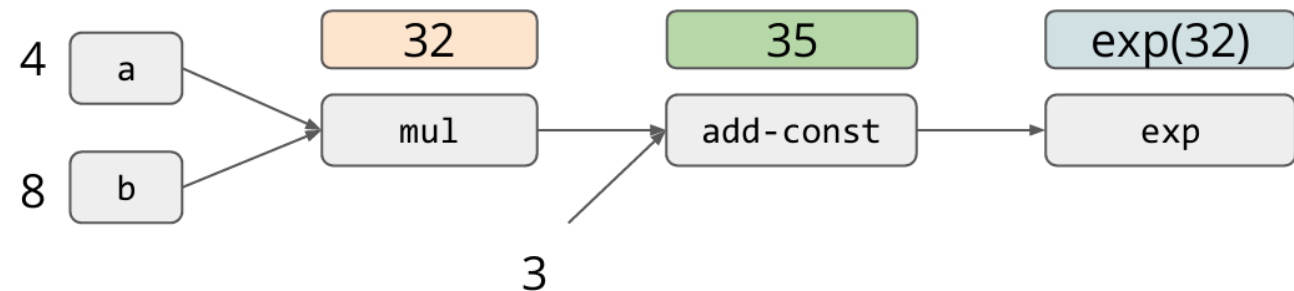
Same color represent same piece of memory



1. **Allocate** temp memory for intermediate computation
2. **Traverse and execute** the graph by topo order.

**Temporary space linear
to number of ops**

Computational Graph for $\exp(a * b + 3)$

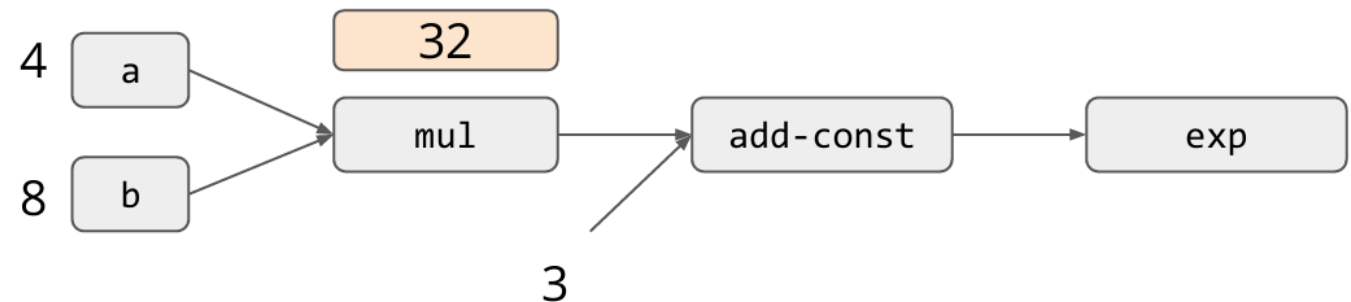


Dynamic Memory Allocation

Static Memory Allocation

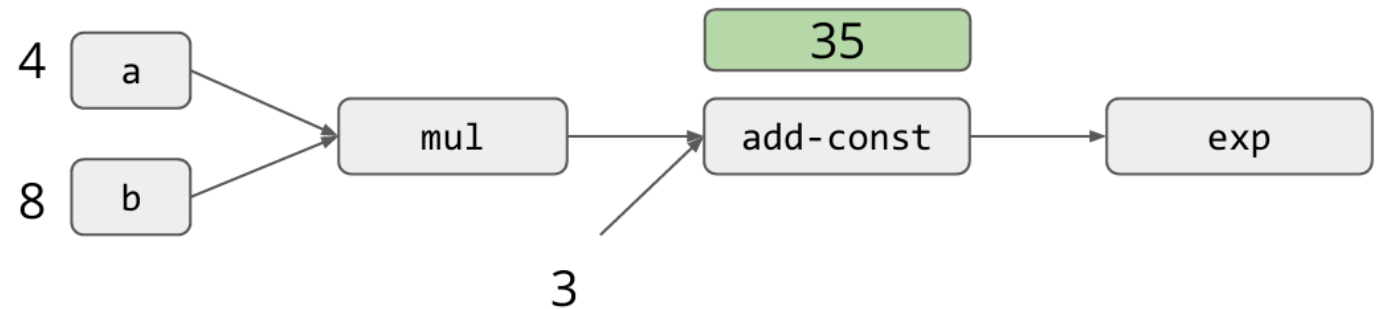
1. **Allocate** when needed
2. **Recycle** when a memory is not needed.

Memory Pool



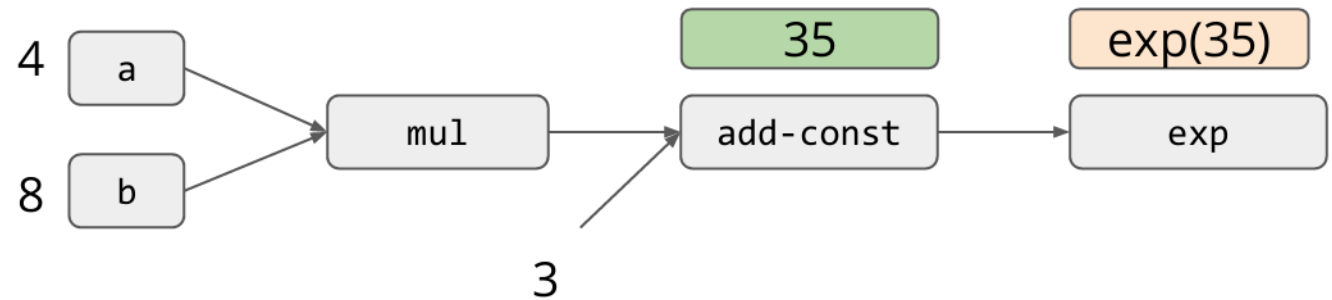
1. **Allocate** when needed
2. **Recycle** when a memory is not needed.

Memory Pool



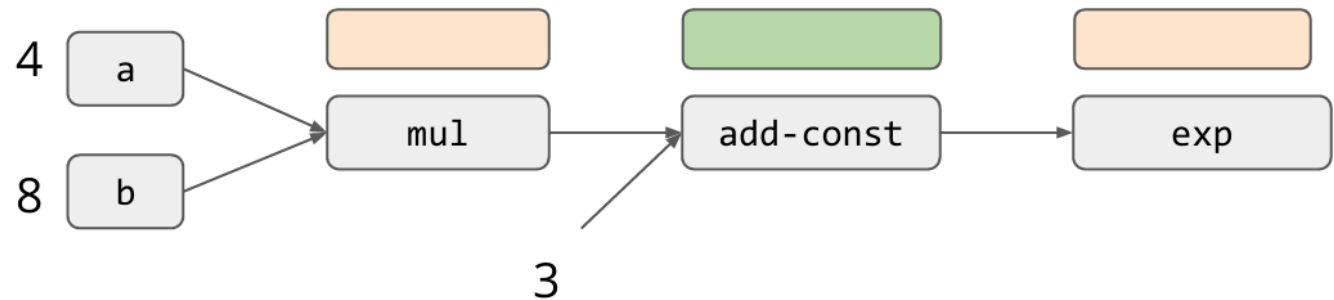
1. **Allocate** when needed
2. **Recycle** when a memory is not needed.

Memory Pool



1. Plan for reuse **ahead of time**
2. Analog: register allocation algorithm in compiler

Same color represent same piece of memory

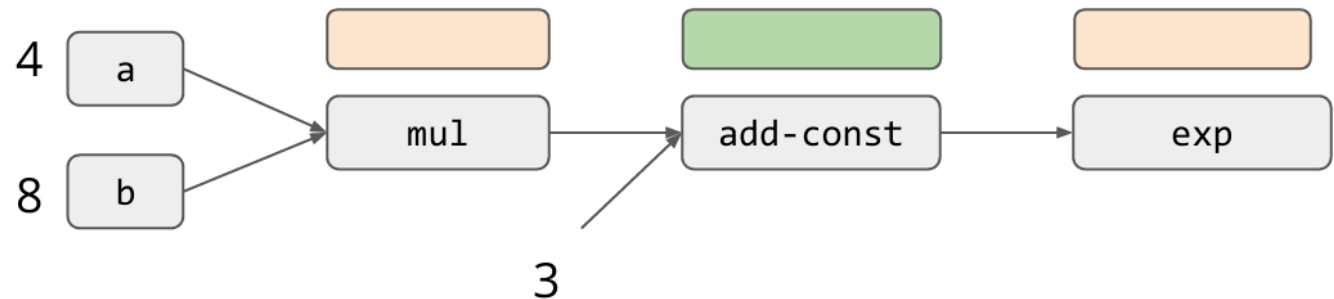


1. Plan for reuse **ahead of time**
2. Analog: register allocation algorithm in compiler

TensorFlow 1.x, PyTorch 2.0 (static graph mode)

Question: What are pros and cons between static and dynamic allocation?

Same color represent same piece of memory



Dynamic Memory Allocation

- Can be done at runtime

Static Memory Allocation

- Determined at compile time

Dynamic Memory Allocation

- Can be done at runtime
- Flexible, more efficient use of memory

Static Memory Allocation

- Determined at compile time
- No runtime overhead

Dynamic Memory Allocation

- Can be done at runtime
- Flexible, more efficient use of memory
- Potential for fragmentation

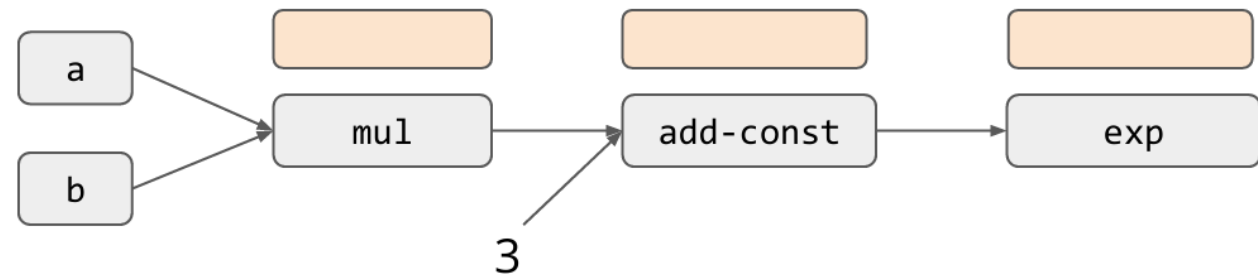
Static Memory Allocation

- Determined at compile time
- No runtime overhead
- Inflexible, can lead to wasted memory

- **Inplace** store the result in the input
- **Normal Sharing** reuse memory that are no longer needed.

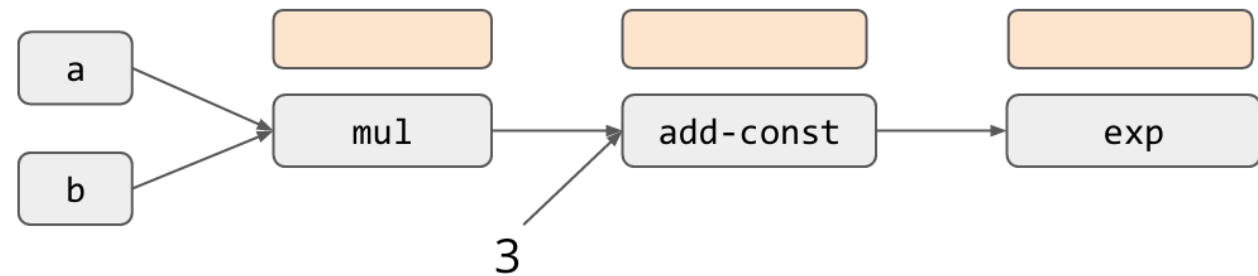
- Operations that modify data directly in the memory location where it is stored
- Examples: In-place activation functions, in-place BatchNorm
- Benefits:
 - Reduce memory footprint
 - Improve computation efficiency

Computational Graph for $\exp(a * b + 3)$



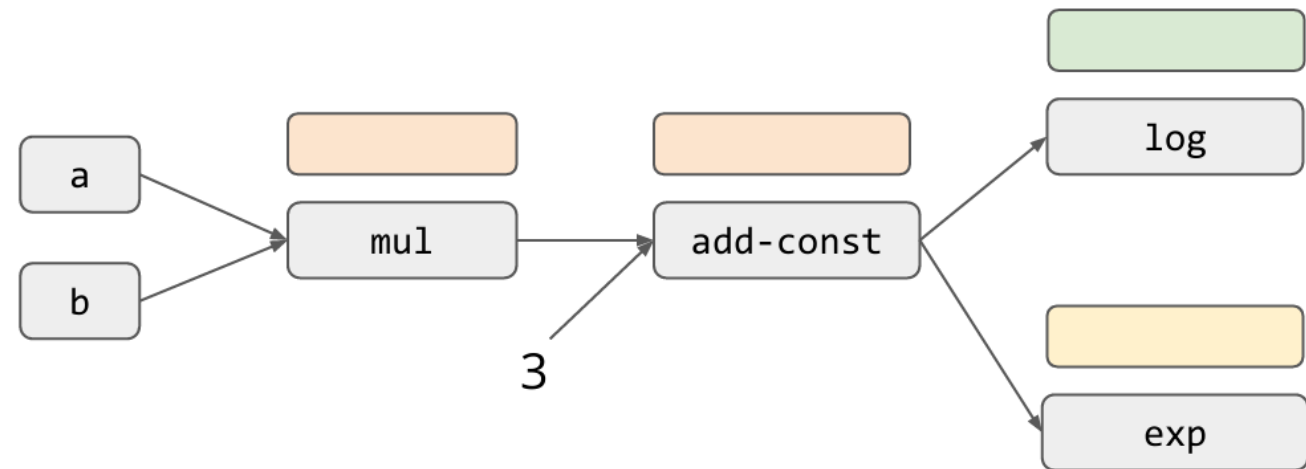
- Operations that modify data directly in the memory location where it is stored
- Examples: In-place activation functions, in-place BatchNorm
- Benefits:
 - Reduce memory footprint
 - Improve computation efficiency

Computational Graph for $\exp(a * b + 3)$



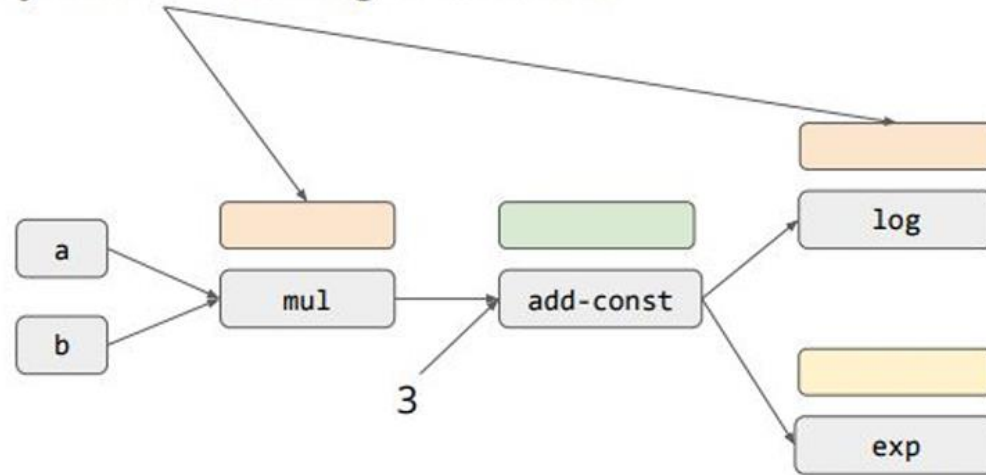
Question: What operation cannot be done in-place?

- We can only do in-place computation if the result op is the only consumer of the current value (i.e., no other operator depends on the current value)



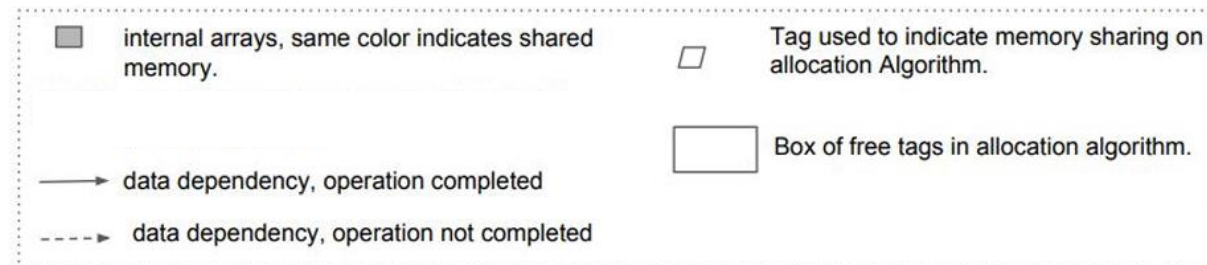
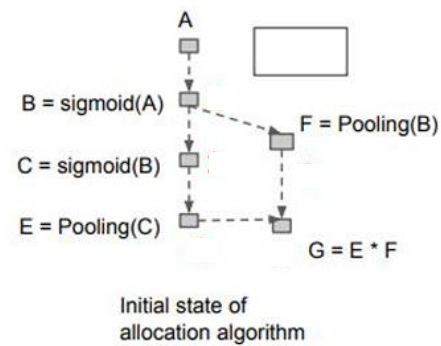
- Memory used by intermediate results that are no longer needed can be recycled and used in another node;
- Shared memory between the nodes whose lifetime do not overlap

Recycle memory that is no longer needed.



- Static Memory Planning: plan for reuse **ahead of time**

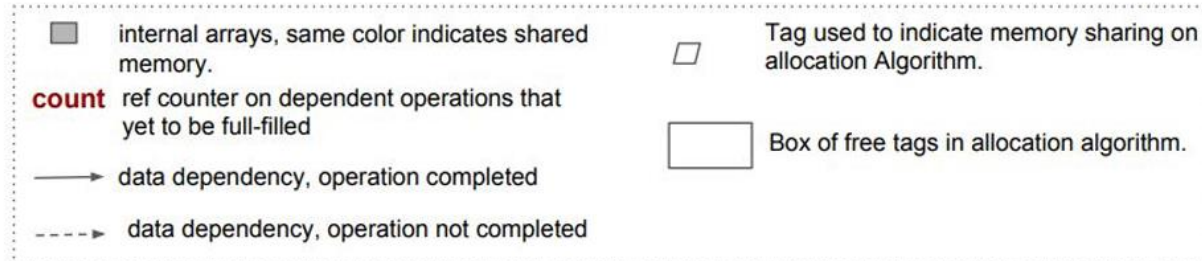
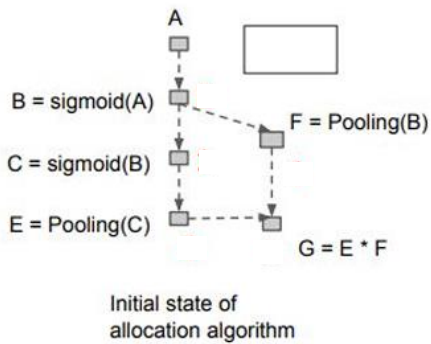
- Static Memory Planning: plan for reuse **ahead of time**



Memory Allocation/Planning Algorithm: Liveness Analysis



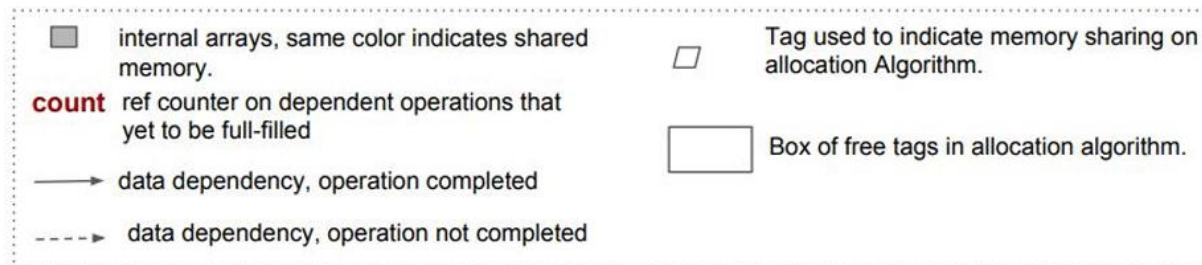
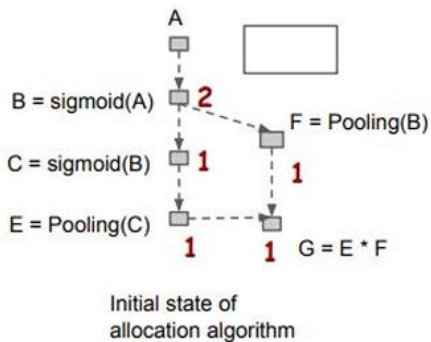
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



Memory Allocation/Planning Algorithm



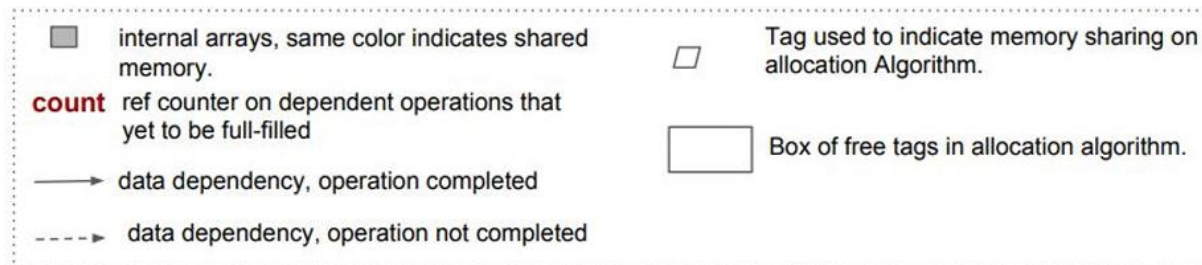
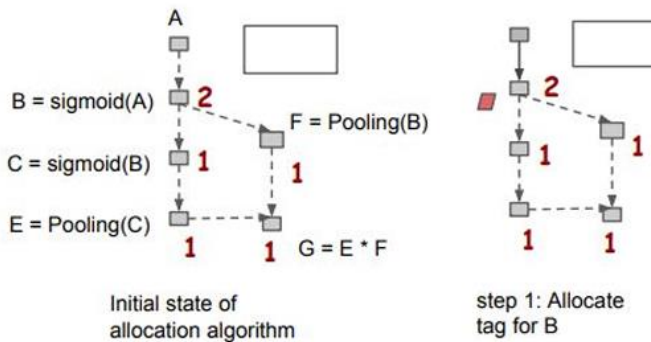
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



Memory Allocation/Planning Algorithm



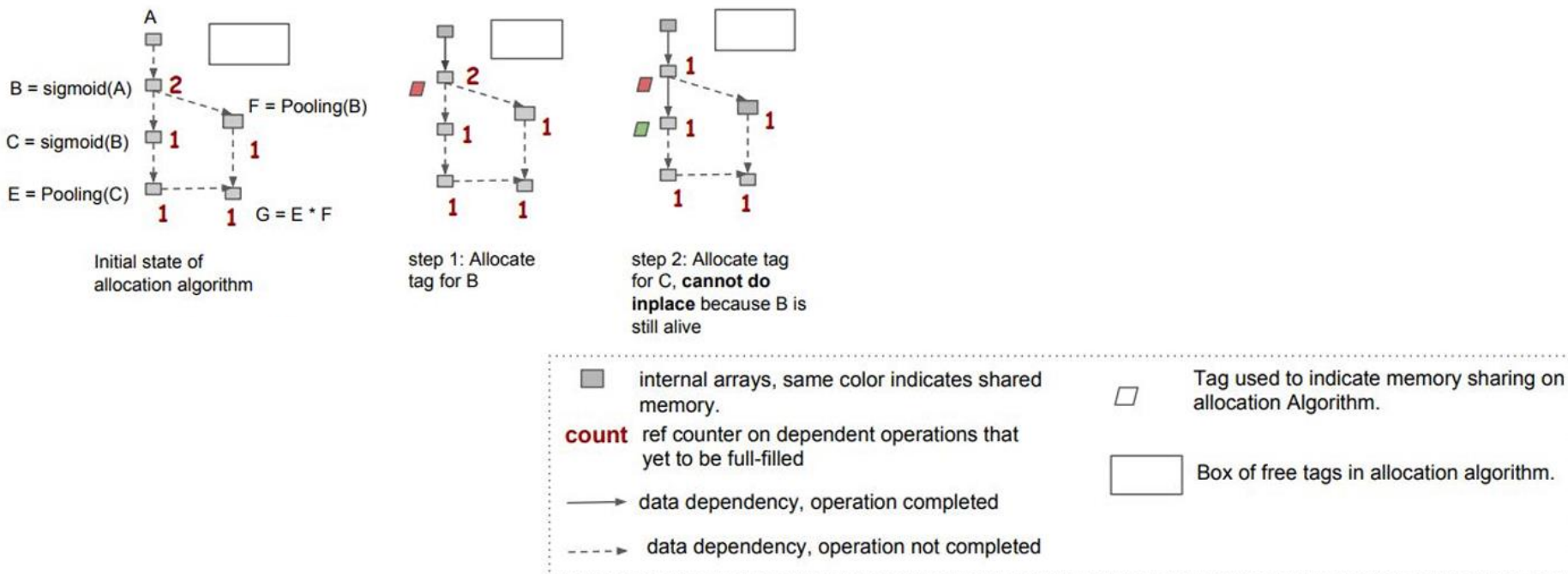
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



Memory Allocation/Planning Algorithm



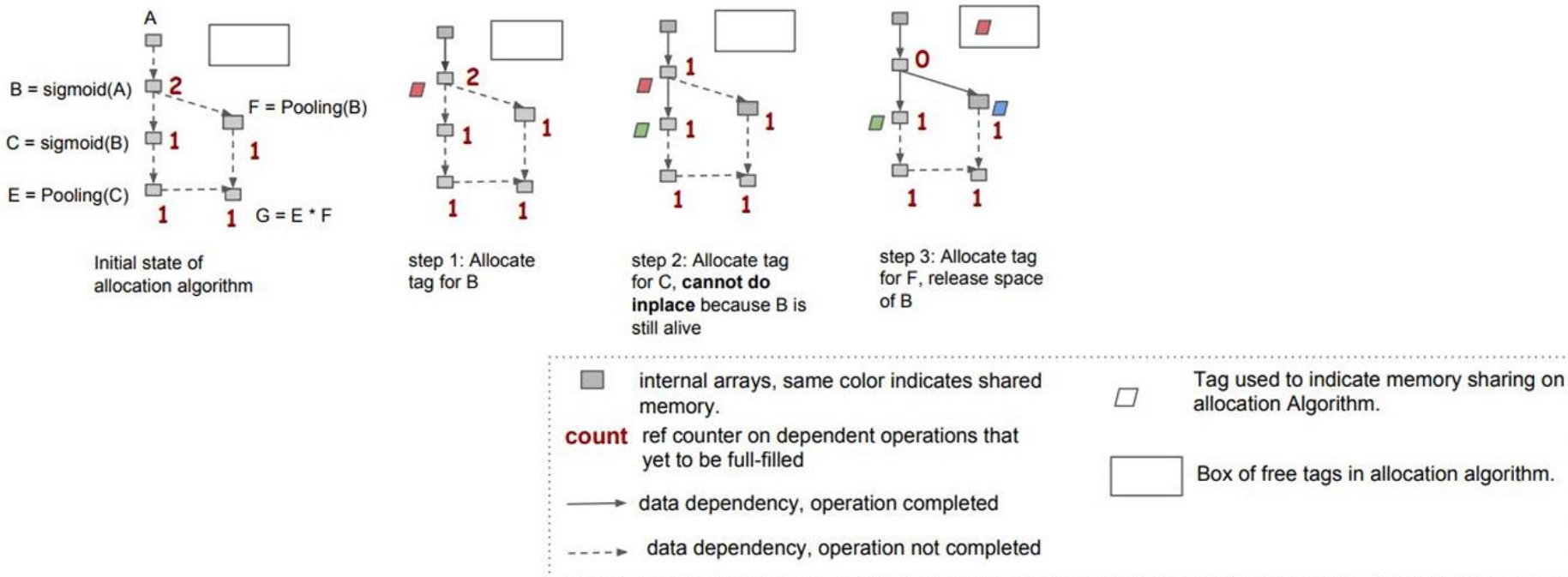
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



Memory Allocation/Planning Algorithm



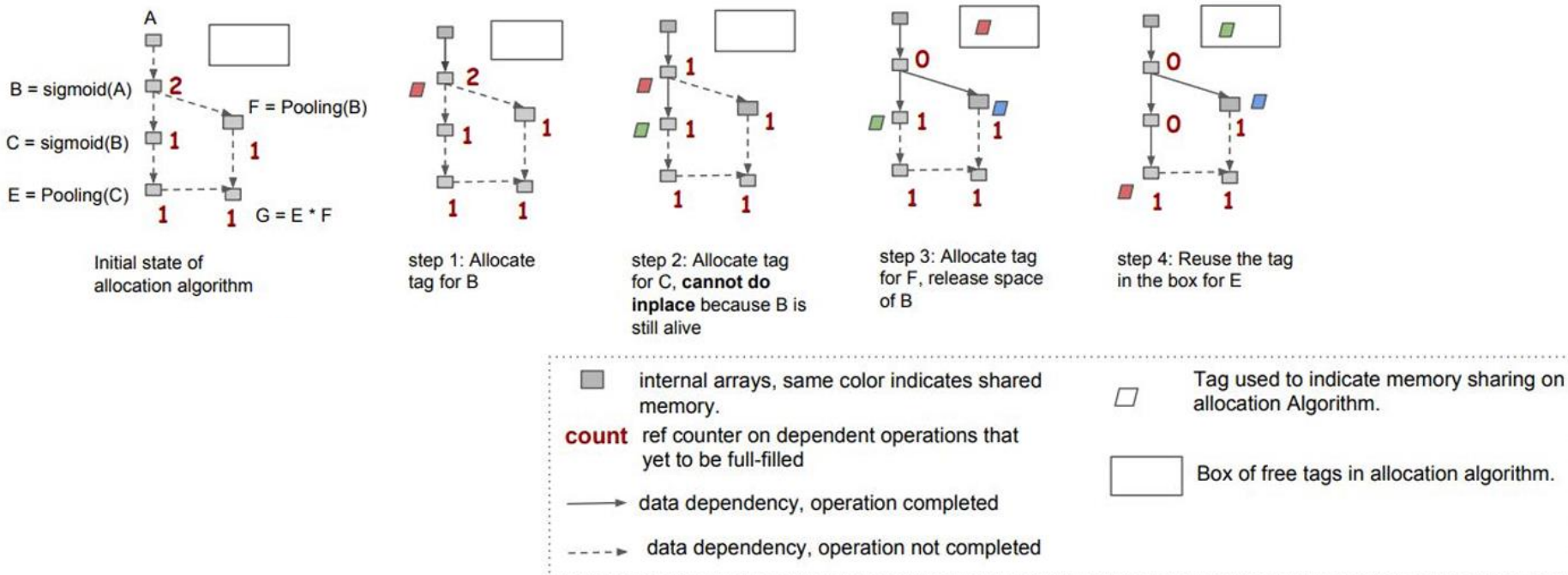
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



Memory Allocation/Planning Algorithm



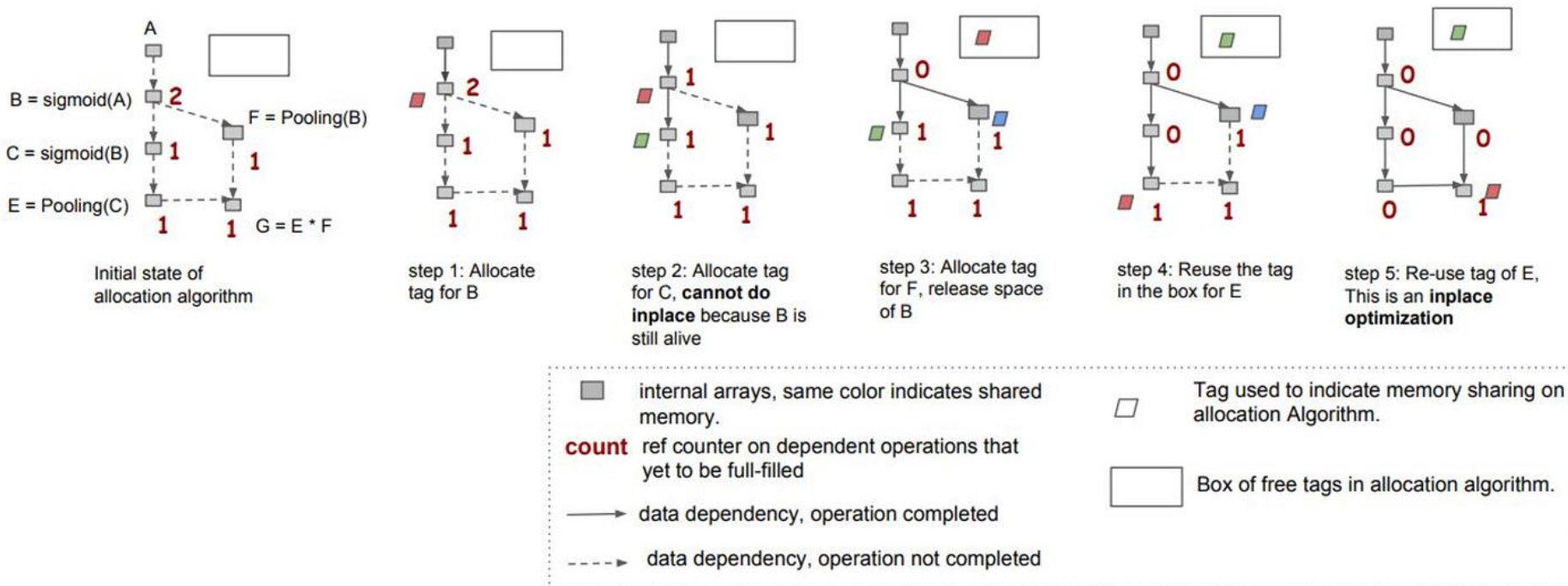
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



Memory Allocation/Planning Algorithm



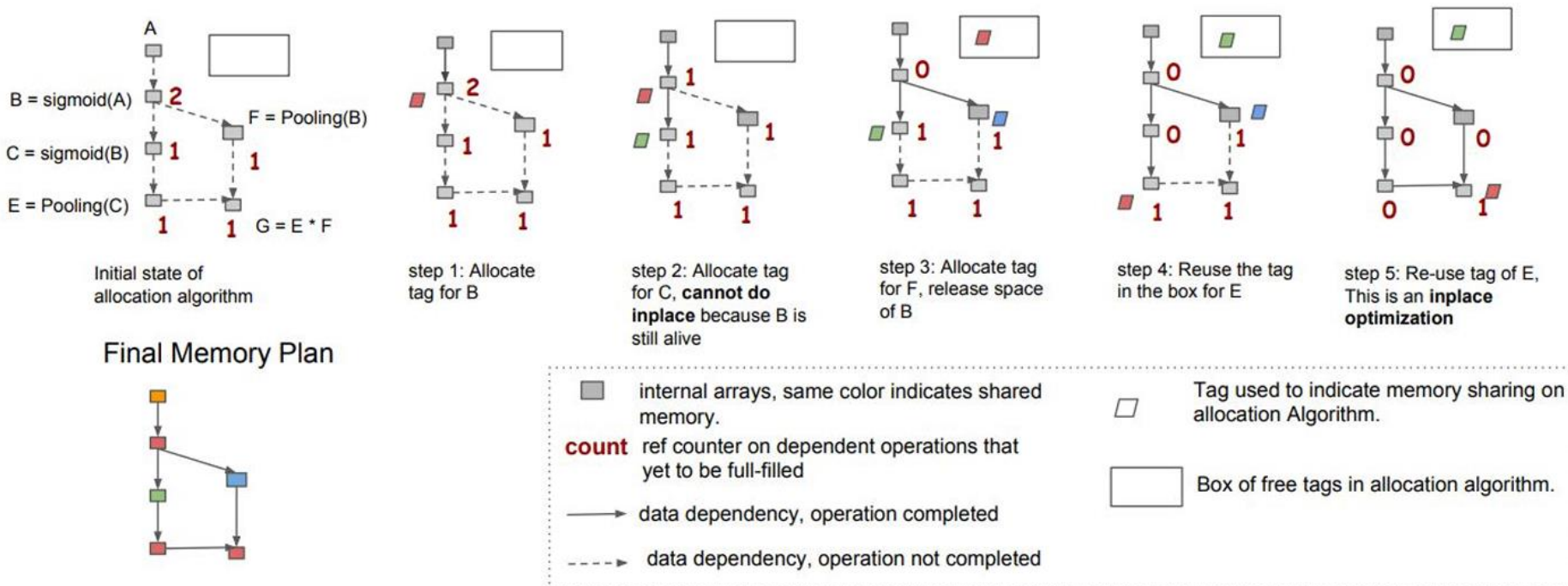
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



Memory Allocation/Planning Algorithm



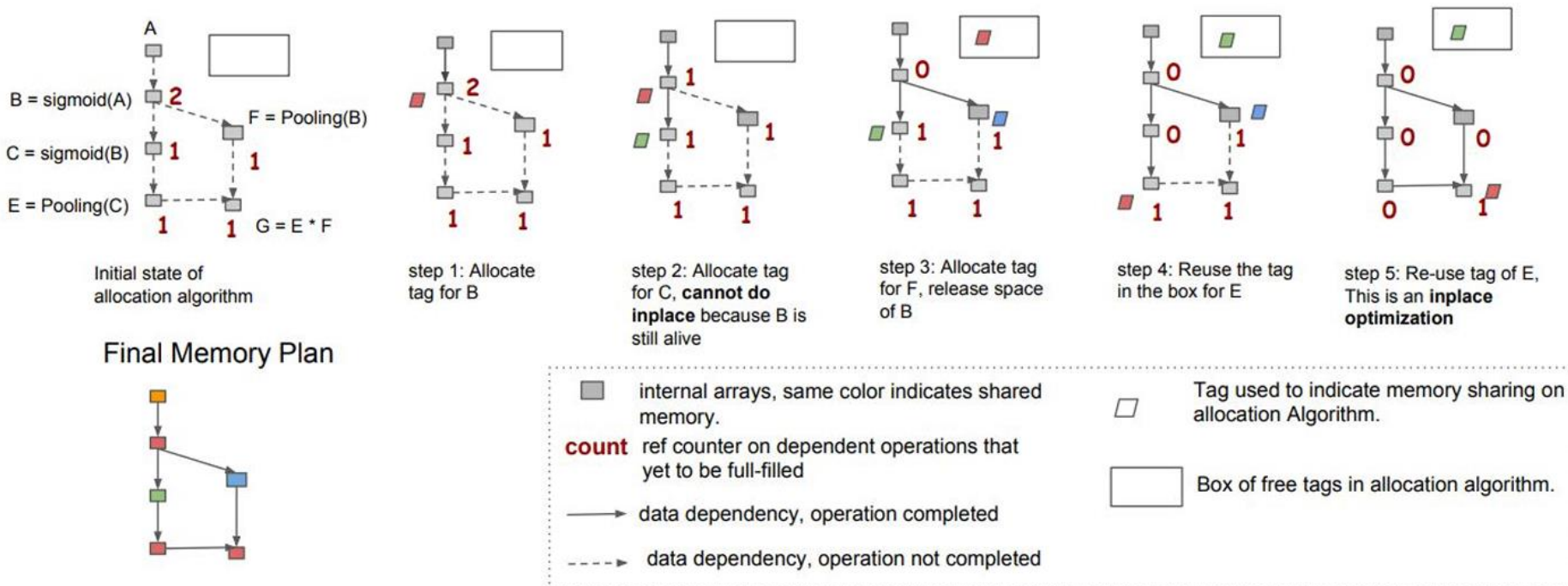
- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness counters**: Track the number of operations that need to be fulfilled before memory can be reused



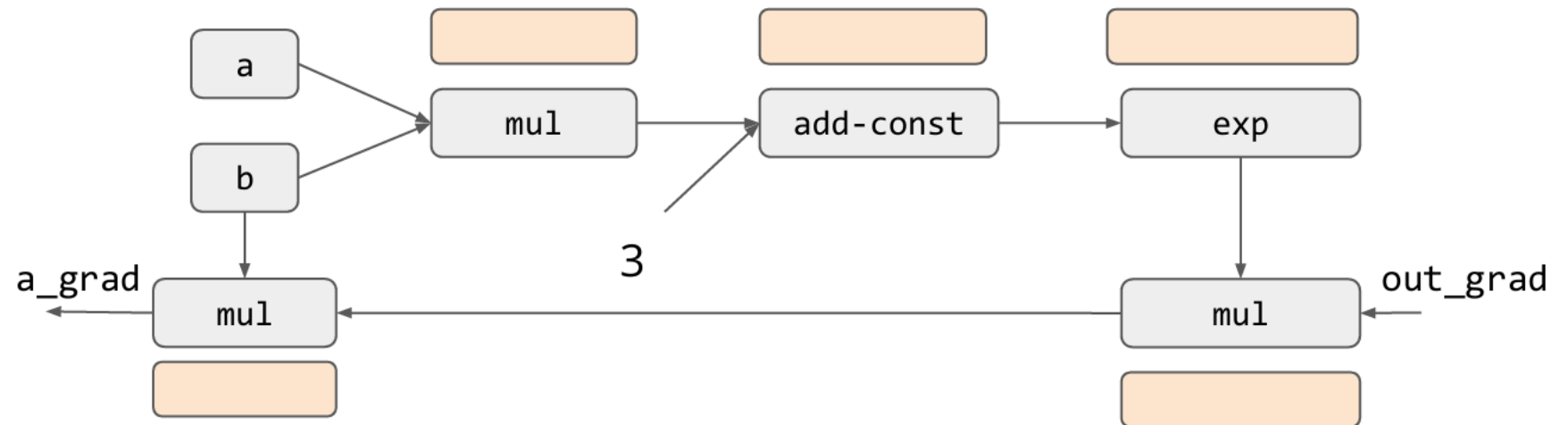
Memory Allocation/Planning Algorithm



- Static Memory Planning: plan for reuse **ahead of time**
- **Liveness** counters: Track the number of operations that need to be fulfilled before memory can be reused
- Dependencies: Important to declare minimum dependencies to optimize memory usage (DL frameworks already handle this for common operators)



We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?



We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?

Idea: Drop **some of** the intermediate results and recover them from an extra forward computation (**from the closest checkpointed results**) **when needed**.



We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?

Idea: Drop some of the intermediate results and recover them from an extra forward computation (from the closest checkpointed results) when needed.

Algorithm:

1. Divide the network into several segments;
2. Store the output of each segment and drops all the intermediate results within each segment;

We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?

Idea: Drop some of the intermediate results and recover them from an extra forward computation (from the closest checkpointed results) when needed.

Algorithm:

1. Divide the network into several segments;
2. Store the output of each segment and drops all the intermediate results within each segment;
3. Recompute dropped results at segment-level during back-propagation

Checkpointing: storing only a few intermediate results during the forward pass.

Recomputation: Recomputing the necessary values during the backward for gradient calculation

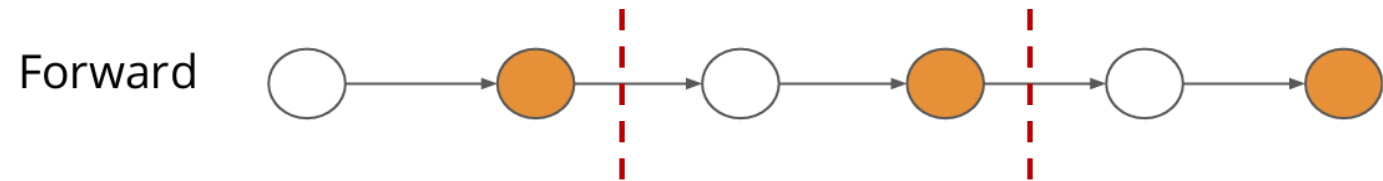


 Data to be checkpointed for backprop

 Data to be dropped

Checkpointing: storing only a few intermediate results during the forward pass.

Recomputation: Recomputing the necessary values during the backward for gradient calculation



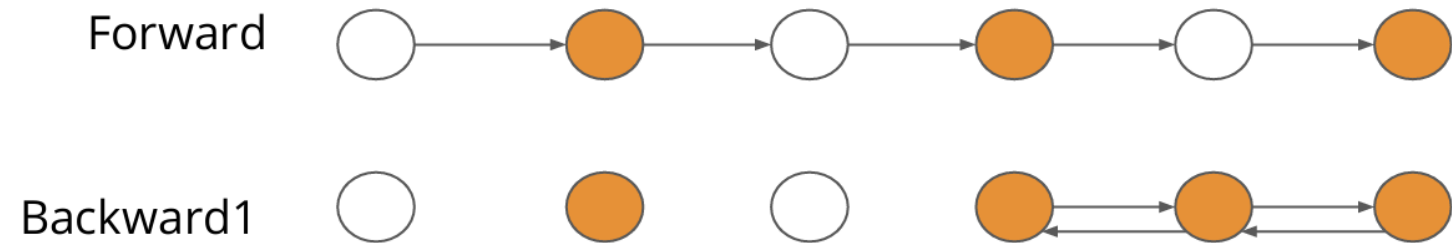
Data to be checkpointed for backprop



Data to be dropped

Checkpointing: storing only a few intermediate results during the forward pass.

Recomputation: Recomputing the necessary values during the backward for gradient calculation

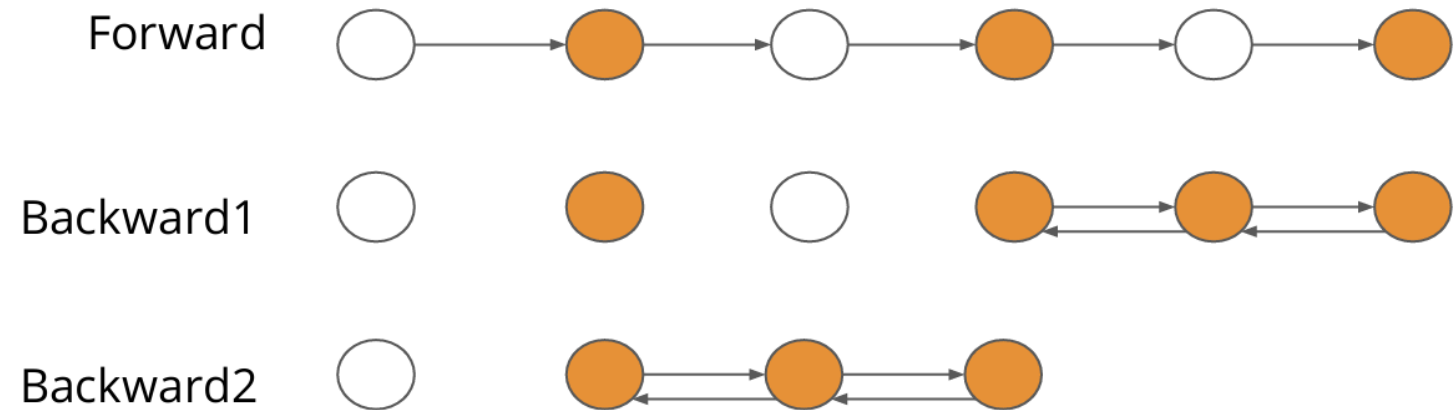


 Data to be checkpointed for backprop

 Data to be dropped

Checkpointing: storing only a few intermediate results during the forward pass.

Recomputation: Recomputing the necessary values during the backward for gradient calculation

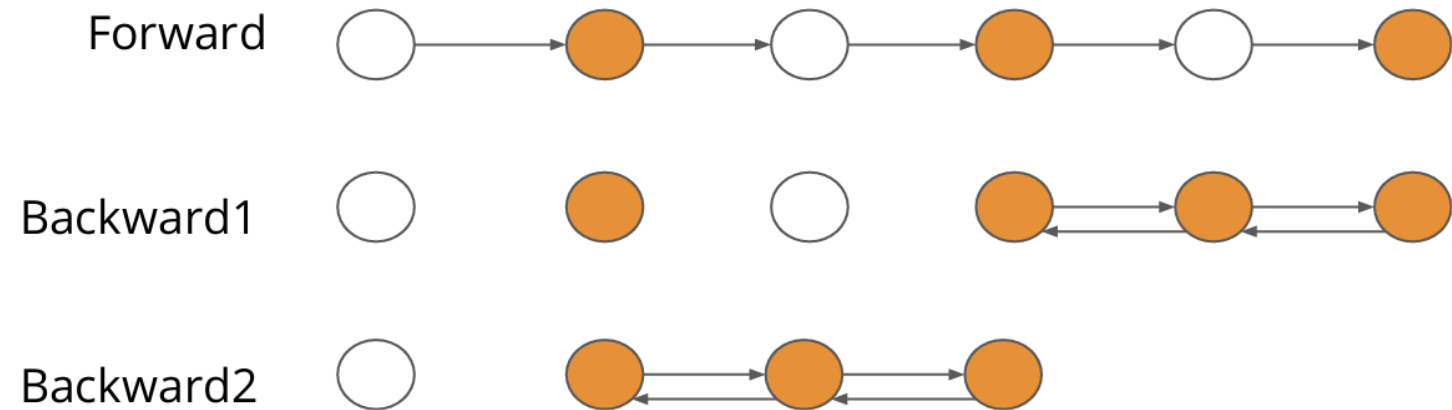


 Data to be checkpointed for backprop

 Data to be dropped

Checkpointing: storing only a few intermediate results during the forward pass.

Recomputation: Recomputing the necessary values during the backward for gradient calculation



Selective Gradient Checkpointing: Drop the results of low compute cost operations, e.g., Conv-BatchNorm-Activation, drop results of the batch norm, activation, and pooling, but keep the results of convolution

Assume we divide the n network into k segments, the activation memory to train the network:

$$\text{cost-total} = \max_{i=1, \dots, k} \text{cost-of-segment}(i) + O(k) = O\left(\frac{n}{k}\right) + O(k)$$

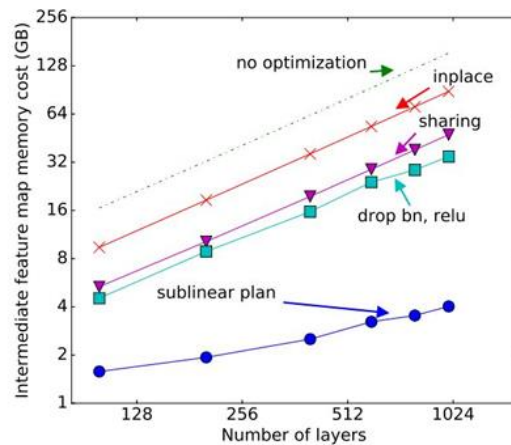
*Max memory demand across
segments*

Assume we divide the n network into k segments, the activation memory to train the network:

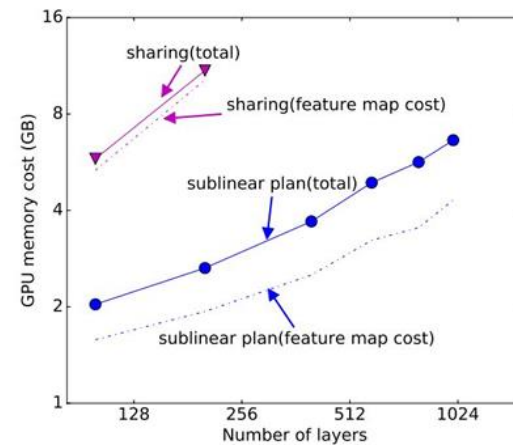
$$\text{cost-total} = \max_{i=1,\dots,k} \text{cost-of-segment}(i) + O(k) = O\left(\frac{n}{k}\right) + O(k)$$

Assume segments are equally divided, setting $k = \sqrt{n}$, we get $O(2\sqrt{n})$ with only one additional forward pass.

Running different schemes on ResNet



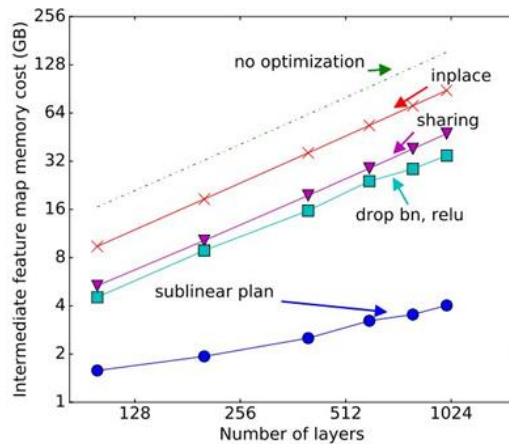
(a) Feature map memory cost estimation



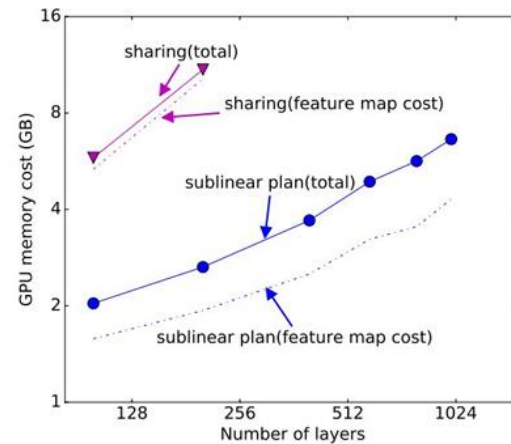
(b) Runtime total memory cost

Training time roughly increases 30%.

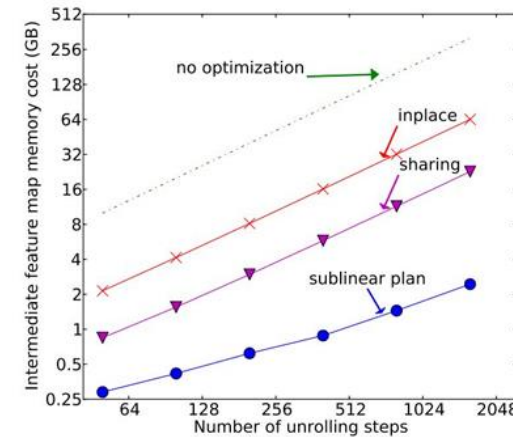
Running different schemes on ResNet and LSTM.



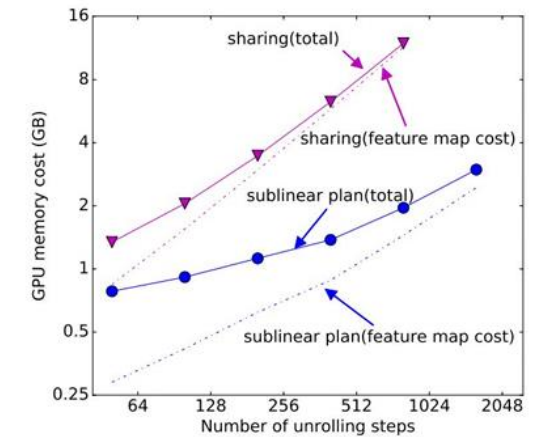
(a) Feature map memory cost estimation



(b) Runtime total memory cost



(a) Feature map memory cost estimation



(b) Runtime total memory cost

Training time roughly increases 30%.

- **Memory management**

- Dynamic Memory Allocation: flexible, efficient but fragmentation
- Static Memory Allocation: no runtime overhead but waste
- In-place allocation and memory sharing (based on liveness)

- **Gradient checkpointing**

- Storing only a few intermediate results during the forward pass
- Recomputing the necessary values during the backward for gradient calculation

Questions?